S8 Custom Soft Processor for Spartan-6 LX9

Technical Architecture, RTL Implementation, Rust Toolchain, USART Bootloader, and W25Q64 Persistence

Target FPGA: XC6SLX9-2TQG144 | Flash: W25Q64 family (exact suffix to be verified) | HDL: Verilog-2001 | Host tools: Rust | FPGA tools: Xilinx ISE

Engineering specification • Implementation roadmap • Verification plan

Document status: implementation specification and step-by-step engineering plan. It distinguishes the Phase 0 starter foundation from modules and behaviors that must still be implemented and verified. It is not a claim that a complete UART/flash-bootable CPU already exists.

1. Executive summary

S8 is a small, custom 8-bit soft processor intended for learning and experimentation on a Spartan-6 LX9 FPGA. The FPGA contains the CPU, instruction/data storage, GPIO, UART, and boot control. A PC-side Rust toolchain assembles source code into an image and transfers that image over a USB-to-UART connection. A W25Q64 SPI NOR flash provides nonvolatile application storage, subject to the critical decision of whether it is dedicated to S8 or shared with the FPGA configuration image.

The recommended order is: verify board and pinout; simulate a minimal CPU; synthesize a GPIO demo; implement UART; implement the upload protocol; verify flash identification and reads; implement robust flash storage; connect the uploader; then build a higher-level compiler. Do not start by implementing every subsystem at once.

2. Goals, scope, and assumptions

- Target FPGA: XC6SLX9-2TQG144. Confirm the exact speed grade and full part marking from the device or BOM; the example name may be XC6SLX9-2TQG144 with a speed-grade suffix.
- External serial interface: a board USB-UART bridge, if actually wired to FPGA user I/O. Verify bridge voltage, RX/TX direction, and the selected FPGA pins from the schematic.
- Application flash: W25Q64-family 64-Mbit / 8-MiB SPI NOR. Verify exact suffix, voltage, package, and datasheet before implementation.
- Mode input: high selects programming mode at reset; low selects application execution. Sample and synchronize the mode input; do not asynchronously switch the running CPU into bootloader state.
- First toolchain release: assembler, image packer, serial uploader, disassembler, and serial monitor. A C-like compiler is a later phase.
- Clock frequency, pin assignments, I/O standards, USB-UART wiring, and flash wiring are board-specific and must not be guessed.

3. System architecture

Layer	Responsibilities
PC / host	Rust CLI, assembler, image packer, uploader, terminal monitor
USB-UART bridge	USB on PC; logic-level UART TX/RX on FPGA
FPGA top	Clock/reset, synchronized MODE, CPU, program/data memory, UART, GPIO, SPI controller
S8 CPU	Fetch → decode → execute; 8 × 8-bit registers; ALU; flags; PC; optional stack
Peripherals	UART registers, GPIO registers, SPI-flash command sequencer
W25Q64	Nonvolatile image slots, metadata, CRC, validity marker

Use one primary FPGA clock domain for the CPU and peripherals where practical. Generate baud ticks and CPU slowdowns with clock-enable pulses rather than creating arbitrary derived clocks in fabric. If a separate clock domain is unavoidable, define explicit CDC synchronizers and timing constraints.

4. Proposed project tree

```
s8-project/
■■■ README.md
■■■ docs/
■   ■■■ architecture.md
■   ■■■ isa-v0.md
■   ■■■ boot-protocol.md
■   ■■■ flash-layout.md
■   ■■■ verification-plan.md
■   ■■■ board-bringup.md
■■■ rtl/
■   ■■■ s8_top.v
■   ■■■ s8_cpu.v
■   ■■■ s8_alu.v
■   ■■■ s8_regfile.v
■   ■■■ s8_program_ram.v
■   ■■■ s8_data_ram.v
■   ■■■ s8_bus.v
■   ■■■ s8_gpio.v
■   ■■■ uart_rx.v
■   ■■■ uart_tx.v
■   ■■■ uart_regs.v
■   ■■■ spi_master.v
■   ■■■ w25q64_controller.v
■   ■■■ s8_bootloader.v
■   ■■■ tb/
■■■ constraints/
■   ■■■ board.ucf
■■■ host/
■   ■■■ s8/                # Rust workspace / CLI
■       ■■■ Cargo.toml
■       ■■■ crates/
■           ■■■ s8-isa/
■           ■■■ s8-assembler/
■           ■■■ s8-image/
■           ■■■ s8-serial/
■           ■■■ s8-disasm/
■           ■■■ s8-cli/
■■■ examples/
■   ■■■ gpio.s8
■   ■■■ uart_echo.s8
■   ■■■ flash_info.s8
■■■ build/                 # generated artifacts; do not commit bitstreams by default
```

5. CPU microarchitecture

5.1 Baseline datapath

- Program counter (PC): 12 bits for up to 4096 instruction words in the first design; increment by one instruction word for fixed-width 16-bit instructions.
- Register file: eight 8-bit general-purpose registers R0–R7. R0 is not hardwired to zero.
- ALU: ADD, SUB, AND, OR, XOR, NOT, compare. Define carry, zero, negative, and overflow behavior explicitly before implementing conditional branches.
- Control unit: simple multicycle finite-state machine or a single-cycle core with synchronous memory semantics accounted for. A multicycle FSM is easier to integrate with block RAM and peripherals.
- Program memory: 4096 × 16-bit words (8 KiB) if the selected BRAM mapping permits; otherwise begin with 1024 × 16-bit words and scale after ISE utilization review.
- Data memory: start small (e.g., 1 KiB) and implement memory-mapped peripheral addresses separately.
- Stack: for CALL/RET, add a bounded stack in data RAM or reserve a hardware return stack. Define overflow/underflow behavior.
- Illegal opcode: halt and expose a diagnostic status, rather than silently executing an arbitrary instruction.

5.2 Clock and reset

Use the board oscillator as the main clock. Add a PERIOD constraint for its real frequency. Implement reset synchronization appropriate to the chosen reset source. For an external active-low reset, asynchronous assertion with synchronous deassertion is a common approach. Do not assume the oscillator is 25 MHz unless the schematic confirms it.

The initial starter used a fabric-generated divided clock as a visible demo convenience. For the production architecture, replace that with a clock-enable counter. A clock enable reduces clock-domain complexity and makes static timing analysis more predictable.

6. ISA v0 and encoding

The starter ISA is deliberately small. A consistent encoding is more important than a large instruction list. The current example encoding uses 16-bit fixed-width instructions, but the early encoding has limitations and must be frozen only after tests cover all instruction formats.

Instruction	Encoding concept	Semantics
LDI Rd, imm6	opcode=1, rd[11:9], imm[5:0]	Rd ← zero_extend(imm6)
MOV Rd, Rs	opcode=2, rd[11:9], rs[8:6]	Rd ← Rs
ADD Rd, Rs	opcode=3, rd[11:9], rs[8:6]	Rd ← Rd + Rs modulo 256
JMP addr12	opcode=4, address[11:0]	PC ← absolute instruction-word address
OUT GPIO, Rs	opcode=5, rs[8:6]	GPIO output ← Rs[3:0]
HALT	opcode=F	Stop instruction retirement until reset

Critical encoding note: LDI's six-bit immediate only loads values 0–63. To load arbitrary 8-bit constants, introduce a second immediate instruction (for example LDIH/ORI) or define a two-word immediate format. Do not let assembler and RTL independently invent instruction encodings.

6.1 Memory map proposal

Address	Register	Read/write behavior
0xF0	GPIO_OUT	R/W; low four bits drive GPIO outputs
0xF1	GPIO_IN	R; synchronized GPIO input samples
0xF2	GPIO_DIR	R/W; bit=1 output, bit=0 input (optional v1)
0xF3	UART_STATUS	R; RX_VALID, TX_READY, error flags
0xF4	UART_DATA	Read pops RX FIFO; write queues TX byte
0xF5	BOOT_STATUS	R; boot mode and image validation status
0xF6	CPU_CONTROL	Privileged/debug-controlled reset or run controls, if implemented

Memory-mapped registers must be decoded outside the RAM address space or within a clearly defined bus fabric. Specify whether writes complete immediately or stall while a peripheral is not ready. A simple first version can use polling status bits instead of interrupts.

7. Hardware modules and responsibilities

Module	Inputs/outputs	Implementation requirements
s8_top	Clock, reset, MODE, UART pins, GPIO, SPI pins	Top-level wiring only; no guessed package pin numbers.
s8_cpu	Instruction word, memory/peripheral bus	PC, state machine, registers, flags, halt/trap.
s8_alu	Operands, operation code	Combinational arithmetic and explicit flag rules.
s8_regfile	Read indices, write index/data/enable	Eight 8-bit registers; synchronous writes.
s8_program_ram	Address, data, write enable	Synchronous BRAM-friendly behavior; CPU must account for read latency.
s8_data_ram	Address, read/write controls	Bounded memory; avoid collisions with MMIO.
s8_bus	CPU bus request	Address decode, wait states, read mux.
s8_gpio	Data, direction, pins	Synchronize inputs; set safe output defaults.
uart_rx	UART_RX, baud tick	Synchronizer, start-bit check, data sampling, stop-bit and framing-error handling.
uart_tx	Byte/valid, ready	Start/data/stop serialization; deterministic ready/valid handshake.
uart_regs	CPU bus, RX/TX interfaces	Status/data registers and preferably small FIFOs.
spi_master	Command bytes, MISO/MOSI/SCLK/CS	SPI mode 0 initially, configurable divider, byte-level handshake.
w25q64_controller	Read/program/erase requests	JEDEC ID, WREN, status polling, bounds, timeouts, page boundaries.
s8_bootloader	UART protocol, flash controller, CPU reset	Receive image, validate, write staging slot, commit valid marker last.

8. Mode selection and boot sequence

The MODE pin is a boot strap, not a continuously acting run-time switch. Synchronize it into the system clock and latch the selected mode after reset. MODE=1 keeps the CPU held in reset and starts the UART programming state machine. MODE=0 validates the stored application image and then starts execution.

```
Power-on / reset
      |
      v
Initialize control state; synchronize MODE
      |
      +---- MODE=1 ----> UART bootloader
      |                    - identify device
      |                    - accept upload packets
      |                    - validate length and CRC
      |                    - program inactive flash slot
      |                    - read back and verify
      |                    - commit metadata last
      |                    - ACK / wait for MODE=0 or RUN command
      |
      +---- MODE=0 ----> Read metadata and validate slots
                           - choose newest valid image
                           - copy image into program RAM, or execute via mapped flash
                           - release CPU reset
                           - application starts at word address 0
```

For the first implementation, copy the selected application from SPI flash into internal program RAM at boot. This makes the CPU fetch path simple and avoids implementing instruction fetch directly over SPI. Ensure image size does not exceed the program RAM capacity. If application images exceed BRAM, design a separate external-memory instruction-fetch interface later.

9. USART electrical and protocol design

9.1 UART physical interface

- Default framing: 115200 baud, 8 data bits, no parity, 1 stop bit (8N1).
- UART is asynchronous; derive a baud tick from the real system clock. For 25 MHz and 115200 baud, the ideal clocks-per-bit is about 217.014; use a phase accumulator or fractional divider to reduce long-term baud error.
- Cross the RX input through a two-flop synchronizer before edge detection and sampling.
- Verify that the USB-UART bridge uses logic-level 3.3 V UART. Never connect RS-232 voltage levels directly to FPGA I/O.
- Use timeouts, framing-error handling, and bounded FIFOs. The bootloader must recover from a dropped byte without writing beyond the target slot.

9.2 Packet protocol v1

Field	Size	Definition
SYNC	2 bytes	Fixed marker, e.g. 0x53 0x38 ('S8')
VERSION	1 byte	Protocol version; initially 1
COMMAND	1 byte	HELLO, BEGIN, DATA, VERIFY, COMMIT, RUN, ABORT, STATUS
SEQUENCE	2 bytes	Monotonic packet sequence number
ADDRESS/OFFSET	4 bytes	Image-relative byte offset or command-specific address
LENGTH	2 bytes	Payload byte count; bounded by packet buffer
PAYLOAD	0..N bytes	Command data
CRC16	2 bytes	CRC over version through payload; polynomial/initial value fixed in protocol document

Define CRC parameters completely: polynomial, initial value, reflection, final XOR, and byte order. A checksum is for transmission-error detection, not cryptographic authenticity. A packet ACK should include command, sequence number, status, and optionally a device-side error code. Duplicate DATA packets should be handled idempotently or rejected predictably.

10. W25Q64 SPI flash subsystem

10.1 Flash geometry and commands

The W25Q64 family is an 8-MiB SPI NOR device. Typical family geometry is 256-byte pages and 4-KiB sectors, but the exact suffix/datasheet controls electrical limits and timing. SPI command bytes commonly used by this design are listed below; verify the exact part datasheet before final RTL.

Command	Opcode	Purpose / required behavior
Read JEDEC ID	0x9F	Read manufacturer/type/capacity bytes and compare against expected family ID.
Read Status Register 1	0x05	Poll BUSY/WIP and inspect write-protect state.
Write Enable	0x06	Issue before program or erase; verify WEL as appropriate.
Read Data	0x03	Opcode + 24-bit address + streamed data.
Page Program	0x02	WREN, opcode/address/data, max 256 bytes without crossing page boundary.
4-KiB Sector Erase	0x20	WREN, sector-aligned 24-bit address, then poll BUSY with timeout.

10.2 Controller state machine

```
IDLE
  -> ISSUE_WREN
  -> SEND_COMMAND_AND_ADDRESS
  -> TRANSFER_DATA (program) OR WAIT_BUSY (erase/program)
  -> POLL_STATUS
  -> CHECK_STATUS / TIMEOUT
  -> VERIFY_READBACK (for writes)
  -> DONE or ERROR
```

The SPI byte engine and the flash command controller should be separate modules. The byte engine handles bit shifting, CPOL/CPHA, divider, and byte-level done. The command controller owns CS assertion across the complete command, command/address ordering, page splitting, WREN, BUSY polling, timeouts, and status errors.

10.3 Storage layout and atomic update

Preferred design: use a dedicated W25Q64 for application images. Example partitioning below is for a dedicated flash only and must not be applied to shared configuration flash without a board-specific partition plan.

Range	Size	Use
0x000000–0x000FFF	4 KiB	Metadata sector or redundant metadata records
0x001000–0x004FFF	16 KiB	Application slot A
0x005000–0x008FFF	16 KiB	Application slot B
0x009000–0x7FFFFF	Remaining	Future images/data/logging

Metadata should include magic, format version, generation counter, image length, image CRC, slot ID, and a validity marker. Write the validity marker last, only after all bytes are programmed and read back successfully. On boot, validate both slots and choose the newest valid generation. Never erase the currently active valid slot until a replacement has been committed.

If the W25Q64 is also the FPGA configuration flash, the FPGA bitstream occupies a region of that device. Do not use the example application addresses blindly. Determine the actual bitstream layout, configuration mode, flash density, and safe user-mode access method from the schematic and Spartan-6 configuration guide. The S8 firmware must never erase or program sectors containing the FPGA configuration image.

11. Rust host toolchain

11.1 Crate structure

```
host/s8/
■■■ Cargo.toml                # workspace
■■■ crates/
    ■■■ s8-isa/               # canonical opcodes and encode/decode
    ■■■ s8-assembler/         # tokenizer, parser, symbols, relocations
    ■■■ s8-image/             # header, CRC, image bounds, .bin/.hex output
    ■■■ s8-serial/            # serial port and packet protocol
    ■■■ s8-disasm/            # instruction decoding for diagnostics
    ■■■ s8-cli/               # `s8 asm`, `s8 upload`, `s8 verify`, `s8 monitor`
```

11.2 Assembler pipeline

- Lexing: remove comments; tokenize labels, mnemonics, registers, numbers, commas, and directives.
- Pass 1: assign each instruction a word address and collect labels/constants. Reject duplicate labels and address overflow.
- Pass 2: parse operands, validate register and immediate ranges, resolve labels, and encode words using s8-isa.
- Output: emit a raw binary in explicitly defined endianness, Intel HEX or line-oriented hex for inspection, and an image manifest containing length and CRC.
- Diagnostics: include file and line number, expected syntax, actual token, and allowed range.
- Tests: known opcode vectors, labels and forward references, malformed syntax, immediate overflow, duplicate labels, address overflow, and encode/decode round trips.

11.3 Command-line interface

```
s8 asm examples/gpio.s8 -o build/gpio.bin
s8 disasm build/gpio.bin
s8 image build/gpio.bin --slot A -o build/gpio.s8img
s8 upload build/gpio.s8img --port COM5 --baud 115200
s8 verify --port COM5
s8 monitor --port COM5 --baud 115200
```

On Linux the serial device may look like /dev/ttyUSB0 or /dev/ttyACM0; on Windows it may be COMx. The CLI should not assume a fixed port. The serial crate should be isolated from protocol logic so tests can use a mock transport.

12. Assembler syntax and sample programs

12.1 GPIO example

```
; GPIO bit 0 high, then loop forever
LDI R0, 1
OUT GPIO, R0
loop:
JMP loop
```

With the starter encoding, the expected words are 0x1001, 0x5000, and 0x4002. These words are a regression test vector for the assembler and RTL. The current 6-bit LDI cannot load arbitrary 8-bit constants; do not use this example encoding as a finished ISA until the limitations are addressed.

12.2 UART echo example (after UART instructions/runtime exist)

```
; Pseudocode until UART register semantics are implemented
wait_rx:
    IN R1, UART_STATUS
    ANDI R1, RX_VALID
    JZ wait_rx
    IN R0, UART_DATA
wait_tx:
    IN R1, UART_STATUS
    ANDI R1, TX_READY
    JZ wait_tx
    OUT UART_DATA, R0
    JMP wait_rx
```

The example uses pseudocode operations not included in the minimal starter ISA. Add IN, ANDI, and a fully specified peripheral operand encoding before attempting to assemble it.

13. Xilinx ISE workflow

- Create a new ISE project and select the exact full FPGA part, including speed grade and package.
- Add RTL files to the design source set. Add testbenches only to the simulation source set.
- Set s8_top as the synthesis top module.
- Create a UCF from verified schematic pin assignments. Add LOC and IOSTANDARD for every top-level port; do not use placeholder pins.
- Add a PERIOD constraint for the actual board oscillator. Check all generated-clock and CDC implications.
- Run behavioral simulation first. Confirm reset, PC, instruction sequencing, GPIO write, halt behavior, and UART/flash state machines in separate testbenches.
- Run synthesis and inspect logic/BRAM utilization. Verify inferred RAM type and read latency.
- Run implementation and timing analysis. Fix timing failures before generating a bitstream.
- Generate the bitstream and program through the board's supported JTAG/configuration method. Keep a known-good recovery bitstream.
- Record ISE version, source revision, constraints, synthesis reports, and bitstream hash for reproducibility.

Use the official Spartan-6 Configuration User Guide UG380 to understand how SPI configuration flash is connected and which configuration pins may become user I/O after configuration. The guide explains that a configuration flash can be accessed after configuration under specific conditions, but this must be designed around the actual board wiring and configuration mode: https://docs.amd.com/v/u/en-US/ug380

14. Verification plan and acceptance criteria

Test	Method	Pass criteria
CPU reset	RTL simulation	PC and registers reset deterministically; CPU does not retire instructions while reset is active.
Instruction vectors	Unit tests + simulation	Assembler words and RTL behavior agree for every supported instruction.
GPIO output	Simulation then scope/LED	Expected output value appears at the verified pin.
MODE sampling	Simulation + hardware	MODE high enters programming state; low boots application; changes during run do not cause unsafe mode changes.
UART RX/TX	Loopback and PC terminal	Correct 8N1 framing, no unexpected dropped bytes at supported rate.
Packet parser	Fuzz/negative tests	Reject bad sync, CRC, length, sequence, and out-of-range offsets without memory corruption.
Flash identification	Read JEDEC ID	Returned ID matches documented chip family.
Flash page boundaries	Write/read tests	Writes split at 256-byte boundaries; no write crosses a page.
Power loss during update	Interrupt power at stages	Old valid slot remains bootable; incomplete slot is ignored.
Image verification	Readback and CRC	Device-computed CRC matches host image CRC before commit.
Timing closure	ISE timing report	All required paths meet the specified clock constraints.
Resource usage	ISE report	Logic and BRAM usage fit the LX9 with margin for later features.

15. Safety, reliability, and failure handling

- Never guess pin assignments. Confirm package pin, bank VCCO, board routing, and peripheral voltage before applying constraints.
- Do not drive FPGA I/O with 5 V unless the exact bank/pin electrical specification explicitly permits it; Spartan-6 user I/O should be treated as voltage-limited.
- Keep UART bootloader and CPU ownership mutually exclusive. Hold the CPU reset while writing an image.
- Bound every length, address, loop, and timeout. Reject any image larger than the program-memory or flash-slot capacity.
- Never perform sector erase on the active image slot. Use inactive-slot updates and commit metadata last.
- Poll flash BUSY with a timeout. Handle WEL/protection bits and read back programmed data.
- Do not erase the FPGA configuration image. If flash is shared, reserve the bitstream region from all S8 operations.
- Provide a recovery route: JTAG reconfiguration and a known-good bitstream should remain possible.
- Synchronize asynchronous inputs and avoid combinational paths from external pins into clock/reset controls.
- Use CRC for transmission and storage integrity. If adversarial uploads matter, add authentication; CRC is not security.

16. Phase-by-phase execution checklist

Phase	Deliverable	Exit gate
0. Foundation	CPU executes built-in program; assembler emits matching words	Simulation passes and GPIO demo is observed.
1. CPU completion	Flags, branches, load/store, defined ISA, BRAM behavior	All ISA tests pass.
2. UART	RX/TX, status/data registers, test loopback	Reliable PC serial exchange.
3. Boot protocol	Framing, ACK/NACK, length and CRC validation	Upload survives malformed packets without corrupting state.
4. SPI discovery	SPI master and JEDEC ID read	Correct flash ID and stable reads.
5. Persistent slots	Erase/program/readback, metadata, A/B slots	Interrupted update leaves previous image bootable.
6. Host uploader	Rust CLI serial upload and verify	Program can be installed without rebuilding FPGA bitstream.
7. Compiler	C-like frontend and code generation	Compiled program matches assembler/runtime tests.
8. Debugger	Disassembly, symbols, status and break support	Repeatable debug workflow.

17. Compiler design (future phase)

Do not begin the compiler until the ISA, assembler, image format, and calling convention are stable. The compiler should use a small, testable pipeline rather than directly translating source strings into opcodes.

```
Source text
  -> lexer
  -> parser / AST
  -> semantic analysis and type checking
  -> intermediate representation (IR)
  -> constant folding / dead-code elimination (optional)
  -> instruction selection for S8
  -> register allocation (R0-R7; reserve registers by ABI)
  -> branch relaxation / label resolution
  -> assembler encoding
  -> image header + CRC
  -> Rust uploader
```

17.1 Minimum language

- Types: initially u8, bool, void; no dynamic allocation.
- Statements: variable declaration, assignment, if/else, while, function call, return.
- Operators: arithmetic/bitwise operators supported by the ISA; explicit semantics for overflow.
- Built-ins: gpio_read(), gpio_write(), uart_available(), uart_read(), uart_write(), delay() only when implemented in the runtime or hardware.
- ABI: document argument registers, return register, caller/callee-saved registers, stack direction, stack limit, and function prologue/epilogue.
- Debug info: map source file/line to instruction word address; initially emit a simple JSON or text symbol map.

18. Known limitations of the starter archive

- The starter CPU is a learning skeleton, not a complete production core.
- The starter top-level uses a built-in demo program; it does not yet load the Rust-produced HEX file.
- UART pins are placeholders; UART RX/TX and the boot protocol remain to be implemented.
- SPI flash controller and persistent storage remain planned components. The byte-level SPI master alone is not a complete W25Q64 driver.

- The starter divider generates a derived CPU clock for the demo; replace it with a clock-enable approach before expanding the design.
- The initial ISA has a six-bit LDI immediate and a limited instruction set; extend and freeze the ISA only with shared definitions and tests.
- The sample UCF must be board-specific and is intentionally not populated with guessed pins.
- Run `cargo test` locally; the document-generation environment did not validate the Rust build or ISE synthesis.

19. Reference documents

- **AMD/Xilinx Spartan-6 FPGA Configuration User Guide UG380** — Configuration modes and SPI configuration flash interface. Essential when W25Q64 may be the FPGA's configuration flash.
  https://docs.amd.com/v/u/en-US/ug380
- **AMD/Xilinx Spartan-6 package pinout files** — Package pin information. Board routing and bank voltage still require the PCB schematic.
  https://www.xilinx.com/support/package-pinout-files/spartan-6-pkgs.html
- **Winbond W25Q64 family datasheet landing page** — Use the exact part suffix datasheet for timing, voltage, status bits, and command support.
  https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?__locale=en
- **Xilinx ISE Design Suite documentation** — Toolchain overview; use the installed ISE version's help for exact supported flows.
  https://www.xilinx.com/products/design-tools/ise-design-suite.html

20. Required decisions before final hardware integration

- Confirm whether the W25Q64 is dedicated to application storage or shared with FPGA configuration.
- Confirm the exact W25Q64 suffix and schematic connections for CS#, CLK, DI/MOSI, DO/MISO, WP#, and HOLD#/RESET# if present.
- Confirm board oscillator frequency, reset polarity, USB-UART bridge part, UART pins, GPIO pins, MODE pin, and bank VCCO.
- Freeze the ISA v1 encoding and binary image format before writing bootloader or compiler code.
- Choose whether application code is copied from flash into BRAM at boot or fetched from external flash. BRAM copy is recommended for v1.

Recommended immediate next action: collect the board schematic or a clear pinout table. Then complete a board-specific UCF and a minimal simulation/synthesis project before wiring the SPI flash controller.